

JARVIS CORE v2

Cloud-First Intelligence + ORVEX-Owned Realtime Voice

Master architecture, complete V1 lessons, development rules, patch workflow, and new-project handoff



Prepared for the clean Jarvis rebuild

Revision: September 15, 2026

New codebase: Jarvis_Core_v2

Initial version: 0.0.1

NON-NEGOTIABLE FALLBACK RULE

KEEP THE CURRENT Jarvis_Real_Time CODEBASE AND CURRENT CHATGPT PROJECT INTACT. They are the V1 fallback, regression reference, and proof of prior behavior. Core v2 is a new build in a new folder/repository and a new ChatGPT Project. Do not overwrite or delete V1.

Contents

1. How the new ChatGPT Project must use this document
2. Executive decision and fallback strategy
3. Product mission and Jarvis north star
4. Final V2 architecture: cloud-first brain, ORVEX-owned voice
5. Intelligence provider abstraction and model-routing rules
6. Reasoning levels, cloud usage, and model escalation
7. ORVEX Voice Engine and full-duplex conversation
8. ORVEX Voice Studio and custom/cloned voices
9. Unified conversation context and referent resolution
10. State machine, event bus, and cancellation model
11. Wake, sleep, personality, response modes, and human interaction
12. Memory 2.0 and multi-speaker identity
13. Accounts, devices, sync, and recovery
14. Billing, entitlements, quotas, and trusted cloud authority
15. Tool gateway, permissions, confirmations, and security
16. Capability modules and V1 behavior parity requirements
17. Autonomous tasks and durable background work
18. Browser/web authentication behavior
19. UI, orb, visual states, and product identity
20. Observability, latency, cost, and reliability
21. Testing, behavioral regression, and release quality gates
22. Exact patch ZIP/apply/test/repair/cleanup/commit workflow
23. Versioning, Issues folder, checkpoints, and device switching
24. Repository structure and environment setup
25. Git setup and first local commands
26. Core v2 phased roadmap
27. V1 milestone history and known lessons
28. Anti-patterns and non-goals
29. Definition of success for Jarvis 1.0.0
30. Exact new ChatGPT Project setup and first-message template
- Appendix A. V1 fallback state
- Appendix B. Patch cleanup command examples
- Appendix C. Regression scenario inventory
- Appendix D. Current model-direction notes and future local-provider option

1. How the new ChatGPT Project must use this document

Instruction to the assistant in the new project

Treat this PDF as canonical unless Tanner explicitly changes a rule. Do not guess project state when a current checkpoint, repository file, test, or V1 reference is available.

- **Read this document first.** Before proposing architecture, code, version milestones, or patches, understand the full product and workflow.
- **Inspect the current Core v2 checkpoint before implementation.** Never assume files from a prior execution environment still exist.
- **V1 is read-only reference.** Jarvis_Real_Time and the current ChatGPT Project must remain intact unless Tanner explicitly chooses otherwise.
- **Challenge weak choices.** Do not reflexively agree. Compare tradeoffs seriously; Tanner remains the final decision maker.
- **Deliver complete artifacts.** Do not paste large implementation files into chat. Use downloadable patch ZIPs or complete requested archives.
- **One focused milestone at a time.** Do not silently begin the next version before the current candidate passes live acceptance and is committed, unless Tanner explicitly asks to continue.
- **Never invent success.** Tool actions, permissions, payments, messages, calendar changes, provider calls, background jobs, and tests must be verified before claiming success.
- **Central context wins.** When wording such as "pause it" or "resume it" is ambiguous, use the authoritative conversation context rather than whichever subsystem happens to react first.

Required response pattern after every delivered update

1. State the delivered version and concise user-visible change.
2. Provide one downloadable patch/archive link, not scattered code snippets.
3. Give exact extract/apply commands or actions.
4. Give the exact automated test command and report the pre-delivery result.
5. Give a short live/manual test sequence with expected results.
6. Give the exact Jarvis start command.
7. State the next recommended single milestone, but do not begin it before acceptance/commit unless explicitly instructed.

2. Executive decision and fallback strategy

DECISION

Build Jarvis Core v2 clean in a new code folder/repository and new ChatGPT Project. Keep V1 runnable. Reuse proven implementations deliberately; do not perform a deep in-place migration and do not throw away V1 lessons.

Why this still holds after the Luna-first change: the launch-critical local-model manager is no longer required, but the foundational rewrite is still justified by the ORVEX Voice Engine, provider-independent intelligence, unified context/state, cleaner tool gateway, and the desire to remove old Realtime-specific assumptions before public launch.

Preserve / reuse deliberately	Rebuild clean
V1 behavior rules, regression tests, permission semantics, proven executors and aliases	Conversation/context core, event/state ownership, referent resolution
Gmail/calendar/browser/computer implementations where cleanly isolated	IntelligenceProvider interface and cloud model router
Accounts, auth, billing concepts and cloud-authority rules	ORVEX Voice Engine, endpointing, AEC, interruption, STT/TTS abstraction
UI assets, orb direction, successful interaction patterns	Typed tool contracts, cancellation model, observability/latency tracing
Patch workflow, issues discipline, testing/live acceptance	New repository organization and clean dependency boundaries
V1 known failures as regression scenarios	Anything tightly coupled to legacy routing/provider assumptions

3. Product mission and Jarvis north star

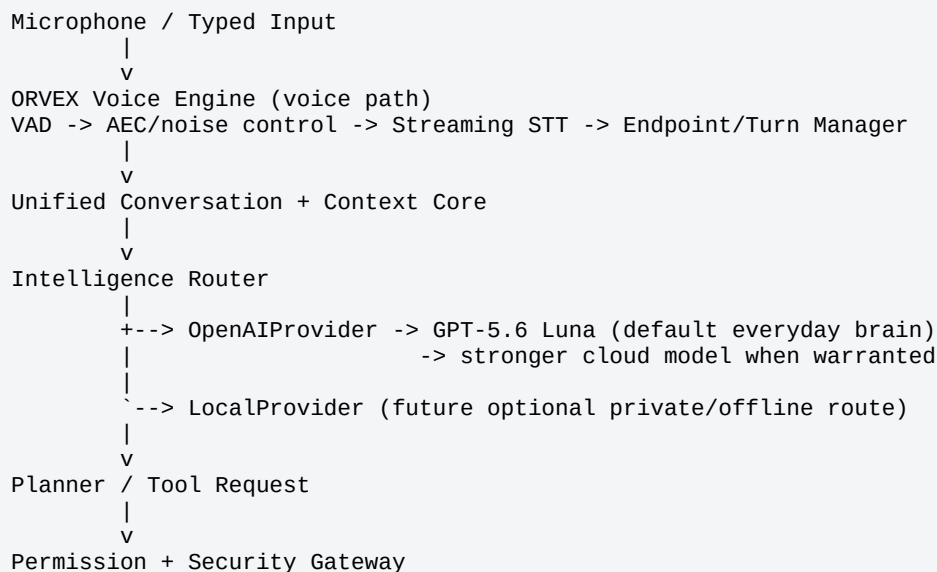
Jarvis is not a chatbot in a desktop window. Jarvis is a persistent personal AI operating layer: one identity, one conversation, one memory system, one permission authority, and many replaceable intelligence/voice providers and capability modules.

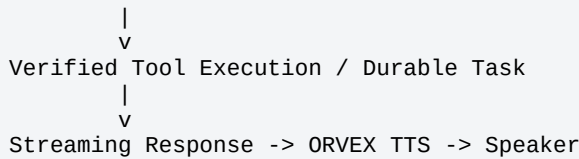
- **Human interaction first.** Fast turn-taking, natural follow-ups, interruption, context-aware references, expressive speech, and concise acknowledgement during long work.
- **Action over chat.** Jarvis should understand, plan, ask for approval when required, execute through tools, verify results, and report naturally.
- **Cloud-first launch intelligence.** GPT-5.6 Luna is the initial everyday brain because it gives consistent capability across desktops, laptops, and future mobile devices without large local-model requirements.
- **Provider-independent identity.** Luna, OpenAI, a future provider, or a future local model is not Jarvis. Jarvis is the orchestration, persona, context, memory, tools, permissions, voice experience, and durable state.
- **ORVEX-owned realtime voice.** Own the audio orchestration, turn manager, interruption behavior, state, streaming, and voice-provider abstractions. Existing speech models may be used underneath.
- **Security outside the model.** Intelligence providers request actions; only Jarvis Core authorizes and executes them.
- **Zero-config install.** Normal users should not install LM Studio, configure model servers, edit JSON, or understand providers.
- **Account-gated product.** Users sign into Jarvis; device identity, subscription, cloud usage, and approved sync are tied to the account.
- **Future local option, not launch dependency.** Keep a LocalProvider interface so private/offline intelligence can be added later without a rewrite.
- **Always-on durability.** Tasks, memory, jobs, logs, compaction, recovery, and background state must survive normal UI/provider restarts.

Long-term capability direction

- Natural voice and typed conversation sharing one context.
- Computer/app control, browser control, Gmail, Calendar, files/folders, media, developer assistance, and future integrations.
- Durable autonomous tasks that can run, pause, resume, cancel, and report progress.
- Comprehensive local/account memory with speaker awareness and unified activity timeline.
- Vision/screen awareness and future mobile endpoints.
- Consumer personal assistant first, with business/developer organization capabilities later behind the same permission/security architecture.

4. Final V2 architecture: cloud-first brain, ORVEX-owned voice





- The UI does not own conversation truth; it renders authoritative core state.
- Voice and typed input enter the same conversation/context system.
- Provider switching must not reset Jarvis personality, memory, active referents, pending approvals, or task state.
- Tool execution is separate from model reasoning and always passes through permissions/security.
- OpenAI-specific objects/endpoints remain inside OpenAIProvider. No business logic may depend directly on Luna-specific schemas.
- LocalProvider is an interface and future roadmap item, not a launch requirement.

5. Intelligence provider abstraction and model-routing rules

5.1 Provider contract

```

IntelligenceProvider
- stream_response(context, tools, reasoning_policy, cancellation_token)
- supports_tools()
- supports_vision()
- supports_reasoning_levels()
- context_limits()
- cancel(request_id)
- health()
- provider/model metadata for logs (not user-facing by default)
  
```

- Core modules call the provider interface, never a provider SDK directly.
- Provider configuration owns model names, endpoints, snapshots, context limits, rate limits, and feature flags.
- Pin/test model versions for production when available. New model versions must run the Jarvis benchmark before default upgrade.
- Fallback is policy-driven. Provider failure must produce a degraded state, not silent duplicated actions.
- Never retry a consequential tool call merely because an LLM request retried. Separate reasoning retry from action idempotency.

5.2 Initial launch routing

Situation	Route
Normal conversation, normal tool planning, common requests	GPT-5.6 Luna through OpenAIProvider
User chooses Quick/Standard and task is ordinary	Luna with appropriate reasoning budget
Hard ambiguity, deep planning, very complex coding/analysis, repeated local/provider failure	Escalate to a stronger configured cloud model if plan/usage allows
High/Extreme selected	Use larger reasoning budget and/or stronger model according to policy
Cloud provider unavailable	Degraded mode; preserve state, avoid duplicate actions, explain limitation
Future Private/Offline mode	LocalProvider when implemented and explicitly selected/allowed

5.3 Current Luna rationale (as of September 15, 2026)

GPT-5.6 Luna is the current default candidate because OpenAI positions it for cost-sensitive, high-volume workloads and it supports tool calling and configurable reasoning effort. Current API list pricing is \$0.20 per 1M input tokens, \$0.02 per 1M cached input tokens, and \$1.20 per 1M output tokens. Treat pricing/model availability as configuration, not hard-coded product assumptions.

Official reference at time of this handoff: OpenAI API model documentation for GPT-5.6 Luna. Re-check current pricing/capabilities before launch or billing decisions.

6. Reasoning levels, cloud usage, and model escalation

User-facing mode	Intent
Auto - Recommended	Jarvis chooses the least-cost/lowest-latency route that should reliably solve the task; escalates when expected value warrants it.
Quick	Prioritize speed and lower usage; minimal reasoning; ordinary Luna route.
Standard	Default balance of reasoning, speed, and usage; Luna remains normal path.
High	Larger reasoning budget and lower threshold for stronger-model escalation.
Extreme	Prioritize maximum available intelligence; may use the strongest allowed cloud model and consume usage much faster.

- Reasoning level never changes permissions or tool authority.
- Do not expose raw tokens to ordinary users. Show a clean monthly AI usage allowance/budget.
- Higher reasoning can consume more output/reasoning tokens and stronger-model usage, so UI must clearly state that High/Extreme use allowance faster.
- Use prompt caching where appropriate for stable system instructions/tool schemas, but never depend on caching for correctness.
- Usage metering should attribute provider/model/reasoning/tool cost per request for observability and future pricing decisions.
- Historical planning example: Basic \$20/month with a \$10 API budget and no rollover. This is not final pricing and must not be hard-coded.

7. ORVEX Voice Engine and full-duplex conversation

VOICE ENGINE PRINCIPLE

The strategic goal is to own the realtime voice experience without training every speech model from scratch. ORVEX owns capture, streaming, turn detection, endpointing, interruption, AEC/noise handling, state, cancellation, routing, and provider abstractions.

7.1 Pipeline

```

Microphone
-> audio device manager
-> noise suppression / acoustic echo cancellation
-> VAD (speech probability)
-> streaming STT
-> semantic endpoint / turn manager
-> ConversationContext
-> IntelligenceProvider
-> response/tool events
-> streaming TTS
-> output mixer / speaker
-> AEC reference signal back to microphone path

```

7.2 Non-negotiable behavior

- Full-duplex/barge-in: detect user speech while Jarvis is speaking.
- On likely interruption, duck/pause outgoing speech immediately; do not wait for full transcription before stopping audio.

- If interruption is real, cancel unplayed speech/text, preserve what the user already heard, and make the new turn authoritative.
- If it was noise, recover gracefully without corrupting conversation state.
- VAD and endpointing are different: silence during a thinking pause must not automatically end the turn.
- AEC must prevent Jarvis from transcribing/responding to his own speaker output.
- STT, LLM, and TTS must stream. Avoid full transcript -> full response -> full audio latency.
- Audio-device changes, Bluetooth reconnects, sleep/wake, game mode, and UI restarts must not corrupt core state.
- Current Cedar voice feel remains the quality reference, but V2 must not depend on Cedar as an architecture.

7.3 Latency instrumentation

Metric	Track
Wake detection / acknowledgement	Wake phrase detection to acknowledgement start
Endpoint latency	Last user speech to confident turn end
STT timing	First partial, stabilized partial, final transcript
Intelligence timing	Request start, first provider token/event, provider complete
TTS timing	First text chunk, first audio, completion
Interruption latency	User speech detected to Jarvis audio stop
Tool timing	Permission request, tool start, verification, completion
End-to-end	User stops speaking to Jarvis first audible reply / verified action

8. ORVEX Voice Studio and custom/cloned voices

- Use a VoiceProfile abstraction so default ORVEX voices, local voices, cloud voices, designed voices, and authorized cloned voices are swappable.
- Voice-profile architecture may store provider/engine, reference/embedding location, style controls, pronunciation dictionary, language, and consent metadata.
- Allow voice cloning only for voices the user owns or has explicit permission to use. Do not design unrestricted public-figure/third-party impersonation as a normal feature.
- Prefer live/randomized enrollment phrases or comparable rights/consent proof when practical.
- Reference audio/embeddings should stay local by default when the engine supports it; encrypted optional sync can be a future setting.
- Benchmark naturalness AND latency, streaming, interruption, pronunciation, memory footprint, licensing, and CPU/GPU behavior.
- Voice tests must include dates, times, currency, percentages, acronyms, names, URLs, short acknowledgements, questions, emotional tone, and interrupted sentences.

```

VoiceProfile
- voice_id / display_name
- engine / provider
- local reference or embedding pointer
- style / pace / warmth / expressiveness settings
- locale / language
- pronunciation hints / dictionary
- rights / consent metadata
- local_only or encrypted_sync
- created_at / updated_at

```

9. Unified conversation context and referent resolution

LESSON FROM V1

Core v2 must have one authoritative context system. YouTube, tasks, browser, voice, and tools must not independently decide what "it", "that", "pause", or "resume" refers to.

```

ConversationContext
- conversation_id / user / speaker
- current_focus_entity
- last_discussed_entity
- recent_referents[]
- active_media
- active_task / active_job
- last_tool_action
- pending_approval
- visible_ui_focus
- temporal context
- working-memory summary
- recent transcript / heard-response state

```

- Explicit wording and current conversational focus outrank stale subsystem state.
- If conversation is about an autonomous task, "resume it" must resume that task rather than an unrelated paused YouTube video.
- When ambiguity remains material, ask one short clarification instead of guessing.
- Context survives voice/typed switching, model escalation, provider failover, UI restart, and task transitions.
- Cancelled/aborted turns must not leave stale pending tool plans as active referents.

10. State machine, event bus, and cancellation model

Core state	Meaning
sleeping	Wake/safety/background functions only; no normal conversation cloud calls.
waking	Initialize minimum audio/core state and prepare active conversation.
listening	Accepting voice/typed input.
thinking	Resolving context, routing provider, planning.
speaking	Streaming TTS/audio and transcript.
executing	Foreground tool/action running.
waiting_for_approval	Consequential action paused for approval/authentication.
background_work	Durable jobs running while normal conversation remains available.
degraded	Provider/audio/tool unavailable and fallback/recovery active.
error	Failure requiring user-visible intervention/recovery.

Representative events should include correlation IDs, timestamps, origin, conversation/user/device/task identifiers, parent-event links where useful, and structured payloads. Examples: user.speech.started, turn.endpointed, intelligence.routed, response.audio.started, response.interrupted, tool.requested, approval.requested, tool.completed, task.paused, task.resumed, provider.degraded, job.progress, memory.write.requested.

Cancellation rules

- Cancellation must propagate from user interruption or explicit cancel to provider generation, queued TTS, pending tool plan, and appropriate task operation.
- Do not cancel already-committed external effects by pretending they never occurred. Verify state and report honestly.
- Consequential actions should use idempotency keys/operation IDs where applicable to prevent duplicate sends/purchases/calendar changes after retries.

11. Wake, sleep, personality, response modes, and human interaction

11.1 Persona

- Jarvis should feel calm, capable, natural, and human - not like a system log.
- Address the owner as "sir" naturally, not every sentence; keep honorific configurable per user.
- Prefer natural wording. Example: instead of "You have one active task...", say "Only one, sir - I need to check your calendar every morning at 8 AM."
- Acknowledge long work quickly, then continue it without blocking normal conversation when architecture permits.
- Do not invent visual details, success, memory, provider results, or tool outcomes.

11.2 Response length modes from V1

Mode	Target
Test	<= 25 words
Short	<= 3 sentences and <= 50 words
Normal	<= 100 words
High	<= 200 words
Full	No hard cap when depth is requested/necessary

11.3 Wake/sleep

- Wake keyword "Jarvis" works inside natural openings and preserves the initial request in the same utterance.
- If the user only says "Jarvis", give a short acknowledgement and wait.
- After waking, follow-up turns do not require the wake word until sleep/timeout.
- Sleep phrases include natural endings such as "that's all" and configurable equivalents.
- Typed chat does not require wake word and shares the same context.
- V1 voice behavior must keep interruption support and avoid getting stuck thinking due to background microphone noise.

12. Memory 2.0 and multi-speaker identity

Memory type	Purpose
Identity/persona	Stable Jarvis behavior, user relationship, configured style.
Entity	People, organizations, devices, projects, accounts, relationships.
Fact/preference	Durable user facts and settings.
Episodic	What happened, when, result/outcome.
Project	Versions, architecture, bugs, tests, next step.
Working	Current task, referents, pending approval, temporary context.
Procedural	How repeated workflows should be performed, including patch workflow.
Document	Searchable files/manuals/contracts/project docs.
Action/audit	Tool requests, approvals, execution results, verification.
Secure vault	Credentials/secrets outside normal promptable memory.

- Write durable state incrementally after explicit decisions, completed interactions, verified tool results, and accepted version changes.
- Deduplicate; keep timestamps, source/provenance, confidence, and superseded values.
- Separate explicit user facts from Jarvis inferences; inferred facts must not silently become high-confidence durable truth.
- Retrieve only relevant memory into prompts; do not dump the full store.

- Use structured source of truth plus exact/full-text and semantic indexes; vector search is not the sole memory database.
- Memory must be speaker/account aware and crash-safe during long uptime.
- Speaker recognition is useful context, not the sole authorization factor; V1 proved exact-original-speaker gating is too brittle for ordinary permissions.

13. Accounts, devices, sync, and recovery

- Account gate remains mandatory before normal use.
- Preserve V1 capabilities/lessons: create/sign-in, recovery, phone validation, lockout, forgot-password flow, verification-code expiry target around 60 seconds, persistent password-visibility behavior.
- Each installation has a device identity under the account. Users should be able to review/revoke devices and invalidate sessions.
- Sync approved durable preferences, selected memory, tasks, aliases, settings, subscription state, and non-secret metadata.
- Do not blindly sync OAuth secrets, SSH keys, browser credentials, arbitrary local files, or entire project folders.
- Privacy-sensitive memory can be local-only or selectively synced.
- New device flow: install -> sign in -> restore approved state -> reauthorize device-specific integrations/permissions -> continue.
- V2 launch intelligence is cloud-first, so normal reasoning requires online connectivity. Future LocalProvider may add private/offline operation.

14. Billing, entitlements, quotas, and trusted cloud authority

SECURITY MODEL

Assume the desktop client can be modified by an attacker. A hacked client may lie to itself, but it must never grant unauthorized cloud/paid privileges or force ORVEX to keep paying provider costs.

- Cloud-authoritative paid privileges. Local snapshots are caches for UX, not financial authority.
- Plans/capabilities/quotas/device limits are configuration-driven through a capability registry; do not scatter tier checks.
- Cloud-paid requests should enforce authentication, entitlement, usage reservation/accounting, provider execution, and settlement in trusted backend boundaries.
- Never ship ORVEX provider API keys, internal pricing/margins, settlement authority, or usage-signing secrets in the desktop app.
- Monthly included usage historically does not roll over; purchased top-ups persist. Treat this as a current design rule unless explicitly changed.
- Cloud usage meters reasoning/provider costs so Quick/Standard/High/Extreme can consume allowance differently.
- Purely local features cannot honestly be made securely tier-exclusive if the entire enforcement boundary is on a hackable client; package/pricing must acknowledge that if LocalProvider ships later.

15. Tool gateway, permissions, confirmations, and security

AUTHORITY RULE

A model may request an action. It does not directly control the OS, accounts, purchases, public channels, or sensitive data. Jarvis Core authorizes, executes, verifies, audits, retries safely, and reports.

Permission level	Behavior
Observe	Read-only inspection under privacy controls.
Safe automatic	Low-risk/reversible action may run without prompting.
Allow once	Approval only for the specific proposed action.
Always allow	Persist permission for that capability/action until changed.

Ask every time	Always prompt.
Authenticated confirmation	PIN/biometric/trusted auth for high-risk operation.
Restricted	Allowed only under context/role/time/mode rules.
Forbidden	Never available to model execution.

15.1 V1 UX/security rules that carry forward

- Permission wording should be simple and human. Preferred buttons: "Cancel", "Allow once", "Always allow".
- Jarvis window must remain in focus when a permission prompt is active. Do not accidentally bring File Explorer/other apps to the front.
- Disabling voice while a permission prompt is visible is acceptable if that avoids accidental spoken approval; the prompt must remain reliable.
- Default should be permissive for ordinary safe actions and restrictive for dangerous/consequential actions, with user settings available.
- After explicit "Always allow", future prompts for that capability should not reappear unless the user changes settings.
- Visible actions can be silently confirmed; hidden actions get a brief spoken confirmation.
- Purchases require a second explicit confirmation of the item and total immediately before execution.
- Full-screen/card controls must be real buttons, not decorative text.
- Do not require exact speaker match for normal permission approval.
- Blocked actions must be explained as blocked; do not ask irrelevant follow-up parameters or misreport provider/tool failure.

16. Capability modules and V1 behavior parity requirements

Module	V1 behavior to preserve/improve
Computer actions	Open/close apps, games, files/folders, aliases, instant index, verified launch. Force-close should be explicit/safe and deterministic when requested.
Browser	Smart search/cards, sources, YouTube control, tab open/close, fuzzy titles, silent confirmations where visible. Browser intent must distinguish web apps from local apps (e.g. "Open Gmail" -> web).
Gmail	Recipient resolution; draft/edit/review/send; scroll/cancel/save; natural confirmations; permission enforcement.
Calendar	Create/update/cancel events, conflicts, attendee add, Gmail follow-up, correct unread/summary phrasing.
Game mode	Game detection, interruption preserved, overlay/snipping behavior without breaking audio/action context.
Accounts/Auth	Create/sign-in/recovery, validation, lockout, code verification, expiry, password visibility control.
Billing/Entitlements	Capability registry, local signed snapshot/cache, device limits, cost tracking, fraud resistance.
Permissions/Security	Persistence, Allow once, Always allow, Block, conversational confirmations, blocked-action preflight, purchase confirmation.
Autonomous tasks	Durable task creation, pause/resume/cancel, status, context-aware references; 0.3.7 remains experimental reference.

Known concrete V1 lessons

- "Pause it"/"resume it" can refer to media or a task; central context must resolve the discussed entity rather than routing the phrase blindly.
- Permission prompts previously lost action context and had speech/noise edge cases; state must be explicit and cancellable.
- After web authentication approval, auto-close the auth tab when possible; if not, show: "Authentication approved, you may now safely close this browser."

- Desktop account sign-in must use the same authoritative account state as web/backend; avoid local credential divergence.
- Calendar delete and other blocked actions must fail cleanly before execution through preflight permission checks.
- Natural phrasing such as "can you open a new tab, please?" must map to the same intent as rigid commands; polish/regression tests should cover natural variants.

17. Autonomous tasks and durable background work

- Tasks are durable objects with IDs, state, schedule/trigger, owner, permissions, budgets, progress, and audit trail.
- Support create, list/status, pause, resume, cancel, retry/recover, and inspect history.
- Conversation remains available while a long task runs. Jarvis should acknowledge immediately and report meaningful progress without becoming robotic.
- "Pause it"/"resume it" must bind to the active discussed task when context says task, even if media is also active.
- Background work must survive normal UI restart and should recover safely after core crash/restart where feasible.
- Autonomous tasks still use the same permission system; autonomy does not bypass confirmations for dangerous/consequential actions.
- Enforce recursion/agent/time/cost limits so autonomous work cannot delegate indefinitely or exhaust usage unexpectedly.

18. Browser/web authentication behavior

- Prefer structured/browser-extension/API integration over brittle coordinate clicking.
- Web authentication approval should auto-close the tab after successful approval when browser security allows.
- If auto-close fails, display exactly or equivalently: "Authentication approved, you may now safely close this browser."
- Browser actions use explicit target/tab identities to avoid acting on stale tabs.
- Opening Gmail should route to the web experience, not try to launch a nonexistent local Gmail app.
- UI cards requiring full screen must expose a real actionable button.
- Sensitive tokens/credentials never enter normal model prompts or logs.

19. UI, orb, visual states, and product identity

The Jarvis orb is a product identity and state display, not merely decoration. The application must expose a clean state/event interface so the orb can evolve from 2D to true 3D without changing conversation or tool logic.

- Current visual direction: highly realistic futuristic orb in blue/white/black with subtle purple accents.
- Outer silver/white armor panels remain separate animatable pieces. In sleep they close/wrap protectively around the outside; in wake they open/expand.
- Core states to visualize: sleeping, waking, listening, thinking, speaking, executing, success, warning/error, interrupted.
- Animation should communicate meaning, not constant random spinning. Speaking should react smoothly to TTS amplitude; thinking should look internally active; interruption should snap attention back quickly.
- Keep ORVEX/Jarvis brand distinction clear. ORVEX is the company; Jarvis is the product. Current product domain: jarvisbyorvex.com. ORVEX.com remains a future acquisition goal if justified.

20. Observability, latency, cost, and reliability

- Every provider request, tool operation, permission decision, task transition, and audio turn should have correlation IDs and timestamps.
- Collect performance metrics without leaking secrets/personal content into logs.
- Measure prompt tokens, cached tokens, output/reasoning tokens, provider/model, latency, retries, and estimated/actual cloud cost.
- Never claim "faster" based only on tokens/sec; optimize mouth-to-ear and request-to-verified-action latency.
- Provider outages/timeouts must not cause duplicate tool execution.
- Crash recovery should reconstruct durable task/permission/tool outcomes from persisted state/audit events.
- Keep diagnostics available for support, but redact tokens/passwords/credentials by default.

21. Testing, behavioral regression, and release quality gates

ACCEPTANCE RULE

Automated tests are necessary but never sufficient. The user must live-test the candidate on the real machine before an accepted commit.

21.1 Standard automated test command

```
python -m unittest discover -s tests -v
```

- Focused regression tests accompany every bug fix.
- Unit tests must NEVER open real applications or perform real external actions. Use `dry_run=True`/mocks/fakes for app launches and tool side effects.
- Add contract tests for provider/tool interfaces, state-machine tests, permission/security tests, integration tests against safe test accounts/fakes, migration tests, soak/restart tests, and performance/latency tests.
- Use real live/manual tests only when the testing guide explicitly instructs the user and risk is understood.
- Collect secret-safe diagnostics when a live test fails; never ask the user to paste secrets or `.env` values.

21.2 Jarvis behavioral benchmark

- Build a corpus of real V1 phrases and edge cases, not just generic model benchmarks.
- Score intent, tool selection, tool arguments, context/referent resolution, naturalness, latency, hallucinated success, failure recovery, permission correctness, and interruption behavior.
- Every new default OpenAI model/snapshot or future provider must run this benchmark before promotion.
- V1 is the behavior oracle where it is known-good; v2 may improve behavior but must not silently regress accepted safety/functionality.

22. Exact patch ZIP/apply/test/repair/cleanup/commit workflow

PATCH DELIVERY STANDARD

This workflow is non-negotiable unless Tanner explicitly changes it. Do not ask the user to manually copy implementation files from chat.

22.1 Every change is delivered as a complete patch ZIP

```
Jarvis_Core_v2<version>_patch.zip
|-- apply_<version>_patch.py
`-- patch_files/
    |-- <new/replacement files in project-relative paths>
    |-- PATCH_NOTES_<version>.md
    |-- TESTING_GUIDE_<version>.md
    |-- JARVIS_CORE_HANDOFF_INSTRUCTIONS.md
    `-- RELEASE_MANIFEST.json
```

- Patch ZIP is extracted into the existing Core v2 project root so `patch_files` merges/appears there.
- Run the root-safe apply script from the project root: `python apply_<version>_patch.py`.
- Do not manually copy individual patch files unless Tanner explicitly asks for an emergency/manual procedure.
- Apply script verifies expected project/root/prior version/checksums where practical before changing files.
- Installer backs up every replaced file with timestamps, preserves rollback, applies atomically where practical, and stops on validation errors.
- Preserve `.env`, provider/OAuth tokens, databases, memories, voice profiles, browser state, logs, Git history, custom `.gitignore`, user files, and unrelated data.
- Installer prints changed files, backup location, and exact next test/start commands.
- Installers should support safe/idempotent reruns when feasible and roll back on failure rather than leaving a half-applied patch.
- Patch documentation must include patch notes, testing guide, current handoff instructions, release manifest, and material Issues documentation.

22.2 Candidate/testing sequence

1. Create/download patch ZIP.
2. Extract ZIP into the Core v2 project root.
3. Activate the Python virtual environment.
4. Run `python apply_<version>_patch.py` from the project root.
5. Run `python -m unittest discover -s tests -v`.
6. Run any focused commands in `TESTING_GUIDE_<version>.md`.
7. Start Jarvis using the exact documented command.
8. Tanner performs live/manual acceptance tests.
9. If something fails, remain on the SAME candidate version using `repair1`, `repair2`, etc. Do not jump to the next normal version.
10. Only after live acceptance: clean patch/backup artifacts, verify git status, rerun final tests if cleanup could affect paths, make one focused commit, then push if a remote is configured.

22.3 Repair naming

Use repair suffixes for failed acceptance corrections (example: 0.0.4-repair1, 0.0.4-repair2). Normal versions increment cleanly only after the current candidate is accepted/committed. Repair labels are not roadmap milestones.

22.4 Cleanup before commit

- Delete the downloaded patch ZIP after it is no longer needed for local rollback.
- Delete `apply_<version>_patch.py` from the project root.
- Delete `patch_files/` after its contents have been applied to their real locations.
- Delete the exact installer-created timestamped backup/staging folder after Tanner accepts and no longer needs local rollback. Git remains the durable rollback.
- Delete explicitly temporary extraction folders/logs that are not intended repository records.
- KEEP intended repository docs and issues/ history.
- Run `git status --short` and verify that no patch ZIP, apply script, backup folder, `patch_files`, `.env`, secrets, databases, model weights, `node_modules`, `.venv`, or unrelated generated files are staged.
- Never use broad wildcard cleanup commands that could delete `.env`, user data, databases, or unrelated files.

22.5 After acceptance

```
git status --short
python -m unittest discover -s tests -v
git add .
git commit -m "0.0.x - Short title"
git push # only when a remote is configured and Tanner wants/preserves remote sync
```

No Git tag by default. Do not create tags unless Tanner explicitly asks to change the established rule.

23. Versioning, Issues folder, checkpoints, and device switching

23.1 Versioning

- Core v2 begins at 0.0.1.
- Increment 0.0.1 -> 0.0.2 -> ... -> 0.0.9 -> 0.1.0. This counting style is preferred.
- Version display/title format: "Version - short title" (for example, "0.0.4 - Conversation Core").
- Promote VERSION to the target candidate during testing so diagnostics reflect what is actually running.
- Every accepted normal version gets one focused Git commit.
- Do not start the next version before acceptance/commit unless Tanner explicitly says to continue.

23.2 Required Issues documentation

Every material issue/update must maintain issues/ documentation covering: symptom, reproduction, root cause, investigation, attempted fixes, final fix, regression tests, verification/live outcome, regression risk, and technical lesson. Do not delete issue history after the bug is fixed.

23.3 Checkpoints and device switching

- When switching desktop/laptop or moving to a new chat, use a complete clean checkpoint from an accepted release or clearly labeled candidate.
- Do not normally include node_modules, .env, build outputs, caches, large model weights, or secrets in checkpoints; restore dependencies from lockfiles/configuration.
- Checkpoint must include VERSION, source, tests, issues, handoff docs, manifests, and everything required to rebuild the working state.
- If a checkpoint is experimental/uncommitted, label it clearly so a new conversation does not treat it as accepted.

24. Repository structure and environment setup

Recommended new code folder/repository name: Jarvis_Core_v2. Exact folders may evolve during 0.0.1; architectural boundaries matter more than cosmetic naming.

```
Jarvis_Core_v2/
|-- apps/
|   |-- desktop/           # Electron/desktop shell + UI
|-- core/
|   |-- conversation/
|   |-- state/
|   |-- intelligence/
|   |-- voice/
|   |-- memory/
|   |-- permissions/
|   |-- tasks/
|   |-- tools/
|-- providers/
|   |-- intelligence/openai/
|   |-- intelligence/local/ # future; may be empty interface at launch
|   |-- stt/
|   |-- tts/
|-- integrations/
|   |-- gmail/
|   |-- calendar/
|   |-- browser/
|   |-- computer/
|-- backend/           # cloud authority/services as architecture evolves
|-- prompts/
|-- tests/
|   |-- unit/
|   |-- contract/
|   |-- behavioral/
|   |-- security/
|   |-- integration/
|   |-- performance/
|-- issues/
|-- scripts/
|-- docs/
|   |-- JARVIS_CORE_HANDOFF_INSTRUCTIONS.md
|   |-- ARCHITECTURE_DECISIONS.md
|   |-- V1_REFERENCE_INVENTORY.md
|-- .env.example
|-- .gitignore
|-- package.json
|-- pyproject.toml / requirements as selected
|-- VERSION
|-- README.md
```

24.1 Do you need a Python virtual environment? YES

Create a local .venv for Core v2. Even if the Electron shell is Node/TypeScript, the ORVEX Voice Engine will likely use Python speech/audio/ML libraries during development. A venv keeps those dependencies isolated and reproducible. Do not commit .venv.

- Recommended compatibility baseline: Python 3.11 unless the selected voice/audio dependency set is verified on a newer version.

- Use one project-local .venv initially for simplicity. If voice/backend services later become separately deployable, split environments deliberately.
- Node dependencies belong in package.json + lockfile; Python dependencies belong in pyproject.toml/lock or requirements files. Never rely on globally installed packages.
- .env holds local secrets/config and is never committed. Commit only .env.example with fake placeholders.

24.2 Minimum .gitignore expectations

```
.env
.env.*
!.env.example
.venv/
venv/
node_modules/
dist/
build/
coverage/
__pycache__/
*.pyc
.patch_backups/
patch_files/
apply_*_patch.py
*_patch.zip
*.log
# add runtime databases, user data, provider caches, model weights, browser profiles as paths are defined
```

Patch cleanup still removes temporary artifacts even when .gitignore would prevent them from being committed. The project root should remain clean.

25. Git setup and first local commands

FIRST-COMMIT RULE

Do not make the first commit until the 0.0.1 foundation structure is created, tested, and accepted. Initializing Git is fine immediately.

25.1 Recommended Windows PowerShell setup

```
cd D:\Github
New-Item -ItemType Directory -Name Jarvis_Core_v2
cd .\Jarvis_Core_v2

git init -b main

# Python environment (recommended)
py -3.11 -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip

# STOP HERE for the clean handoff.
# Do not npm init, add dependencies, or create the first commit until the new ChatGPT Project defines/delivers 0.0.1.
```

25.2 Confirm state

```
git status
python --version
git branch --show-current
```

If Python 3.11 is not installed, install it before creating the venv or explicitly choose the version supported by the selected voice stack. Do not reuse/copy the old V1 .venv into Core v2.

25.3 Remote repository

A GitHub remote is recommended once the initial foundation is accepted. Create a private repository named `Jarvis_Core_v2`, then add it as origin using the URL GitHub provides. Do not expose `.env`, keys, databases, or proprietary provider credentials. The exact remote command should use the user's actual repository URL rather than a guessed URL.

26. Core v2 phased roadmap

Version	Goal / exit criteria
0.0.1 - Foundation	New repo/docs/tests/issues structure; provider/tool/voice contracts; V1 fallback recorded; no legacy copy; app can run a minimal health/diagnostic path.
0.0.2 - Intelligence Provider	OpenAIProvider behind IntelligenceProvider; GPT-5.6 Luna initial default; streaming/tool contract; benchmark harness; no direct OpenAI coupling outside provider.
0.0.3 - Conversation Core	One ConversationContext, referent resolver, state machine/event bus, typed path, cancellation IDs.
0.0.4 - Voice Lab	Audio device layer, VAD, streaming STT, endpointing, streaming TTS, interruption skeleton, latency telemetry; compare against V1 reference.
0.0.5 - Full-Duplex Voice	AEC/noise handling, reliable barge-in, heard/unheard response tracking, audio recovery, voice state polish.
0.0.6 - Reasoning + Usage Router	Auto/Quick/Standard/High/Extreme, usage accounting, stronger-model escalation policy, provider health/fallback.
0.0.7 - Tool Gateway + Security	Typed tool requests, permission engine, audit, one safe end-to-end tool, adversarial tests.
0.0.8 - Memory 2.0	Structured working/durable memory, retrieval, identity/account boundaries, persistence.
0.0.9 - Account/Cloud Authority	Auth/device/entitlement/usage contracts, recovery/sync boundaries.
0.1.0 - Core Vertical Slice	Voice/typed -> context -> Luna/provider -> permissioned tool -> verification -> natural response, stable enough for daily internal use.
0.1.x+	Port V1 capabilities one at a time: computer, browser, Gmail, Calendar, tasks, then UI/orb/vision/mobile/business. LocalProvider remains optional/future unless priorities change.

The ordering can change when testing proves a dependency. The one-at-a-time patch/live-accept/commit discipline does not change.

27. V1 milestone history and known lessons

Milestone	Accepted result / lesson
0.1.0 Desktop Alpha (2026-07-29)	First installed Windows app opened.
0.1.3 Memory Consolidation (2026-07-30)	ORVEX entity accepted; memory direction established.
0.2.1 Gmail Actions (2026-08-10)	Recipient resolution; draft/edit/review/send; scroll/cancel/save; phrasing repairs.
0.2.3 Calendar Intelligence (2026-08-11)	Scheduling/updates/cancellation/conflicts/attendees; Gmail follow-up.
0.2.4 Voice pipeline repairs	Luna authorship/Cedar TTS, prosody/simplification, voice independence and context separation lessons.
0.2.5 Speaker recognition	Owner identity and multi-sample voice confidence; exact speaker must not be sole permission factor.
0.2.6 Computer Actions (2026-08-18)	Apps/games/files/folders, aliases, index, verified launch.

0.2.7 Game Mode & Interruption (2026-08-18)	Game detection; interruption restored; overlay/snipping behavior.
0.3.0 Browser Intelligence	Search cards/sources, YouTube, tabs, fuzzy titles, silent visible confirmations.
0.3.1-0.3.4 Account Foundation	Create/sign-in/recovery/phone/lockout/forgot password/code verification.
0.3.5 Billing & Entitlements (2026-09-01)	Capability registry, signed snapshot/cache, device limits, cost tracking, fraud resistance.
0.3.6 Permissions & Security (committed 2026-09-10)	Foundation/executor enforcement/confirmation security/sensitive hygiene/adversarial testing. Final regressions passed after repairs.
0.3.7 Autonomous Tasks	Next/experimental work. Known referent problem: task resume vs YouTube resume; treat as experimental unless actual Git history proves accepted later.

V1 patch/development conventions inherited

- Patch ZIP extracted into the main project folder; run versioned Python apply script.
- Unit tests command: `python -m unittest discover -s tests -v`.
- Unit tests never open real apps; `dry_run=True` or safe fakes.
- Issues folder required for every material update/issue.
- Repair suffixes only for failed candidate corrections.
- After acceptance delete patch installer/staging/backups before commit; preserve `.env` and user data.
- Checkpoint commits are used when switching machines/chats; clearly label experimental states.

28. Anti-patterns and non-goals

- Do not copy the entire V1 repo and resume patching it under a new name.
- Do not build Jarvis Core around OpenAI/Luna-specific objects. Use provider abstraction.
- Do not make local LLM support a launch blocker; keep the interface for future Private/Offline mode.
- Do not let UI, YouTube, tasks, or browser own separate global conversation state.
- Do not keyword-route "pause/resume" to a subsystem without referent resolution.
- Do not continuously stream microphone/screen/camera to cloud while sleeping or without product need/consent.
- Do not give LLMs unrestricted shell/browser/account/financial access.
- Do not use coordinate clicking as the primary automation strategy when APIs/structured automation exist.
- Do not use vector search as the only memory source of truth.
- Do not hard-code pricing tiers/model names throughout the codebase.
- Do not make reasoning level grant more permissions.
- Do not recursively delegate agents/jobs without time/cost/depth limits.
- Do not train every speech model from scratch just to call the pipeline "ours"; own orchestration and replaceability first.
- Do not build unrestricted voice impersonation.
- Do not commit patch ZIPs, backups, `.env`, secrets, databases, model weights, `node_modules`, `.venv`, or temporary artifacts.

29. Definition of success for Jarvis 1.0.0

- Natural, fast, interruption-capable voice and typed chat share one identity/context.
- GPT-5.6 Luna (or its benchmark-proven replacement) handles everyday intelligence consistently across supported devices; stronger cloud reasoning is available for difficult tasks/selected modes.
- Provider can be swapped without rewriting voice, tools, memory, permissions, UI, or business logic.
- ORVEX Voice Engine owns realtime turn-taking, streaming, endpointing, interruption, and voice-provider abstractions.
- Permissions/security remain authoritative regardless of model intelligence.
- Memory is relevant, deduplicated, account-aware, reviewable, and crash-safe.
- Accounts restore approved synced state across devices without leaking local secrets.
- Billing/usage cannot be bypassed by modifying the desktop to make ORVEX incur unauthorized cloud cost.
- Autonomous tasks are durable, inspectable, pausable/resumable, budgeted, and context-aware.

- Computer/browser/Gmail/Calendar behavior reaches V1 parity or better under the new tool gateway.
- Custom voices can be supported through provider-independent Voice Studio with rights/consent safeguards.
- UI/orb can evolve or restart without disrupting core runtime/background work.
- Important actions, permission decisions, provider routes, cloud usage/cost, latency, and failures are traceable.
- Project continuity works: a fresh ChatGPT Project/chat can resume from repo docs, checkpoint, tests, and this handoff without hidden context.

30. Exact new ChatGPT Project setup and first-message template

30.1 Best setup

1. KEEP the current ChatGPT Project and Jarvis_Real_Time folder exactly as the V1 archive/fallback.
2. Create a new ChatGPT Project named exactly: "Jarvis Core v2".
3. Create a new computer folder/repository: D:\Github\Jarvis_Core_v2.
4. Initialize Git and create the .venv using the commands in Section 25, but do not make the first commit yet.
5. Upload THIS PDF first to the new ChatGPT Project.
6. Upload the latest complete V1 checkpoint ZIP for reference. Prefer a clean committed 0.3.6 baseline; if including 0.3.7 work, label it experimental/unaccepted unless Git proves otherwise.
7. Optionally upload a simple text file containing the current V1 commit hash and any uncommitted state. Do not include secrets.
8. Send the first-message template below.
9. Let the new project define/deliver the 0.0.1 foundation before installing app dependencies or committing.
10. After 0.0.1 exists, use the patch ZIP workflow for every version/repair.

30.2 First message to send in the new ChatGPT Project

We are starting Jarvis Core v2 as a clean rebuild. Read the attached "Jarvis Core v2 - Cloud-First Intelligence + ORVEX-Owned Realtime Voice" master handoff PDF first and treat it as canonical unless I explicitly change a rule.

The existing Jarvis_Real_Time codebase and my previous Jarvis ChatGPT Project are V1 and must remain untouched as my fallback/reference. I am attaching the latest complete V1 checkpoint for behavior/code reference.

Our launch architecture is cloud-first and model-independent: GPT-5.6 Luna is the initial everyday default through an IntelligenceProvider abstraction, with stronger cloud-model escalation for hard tasks or higher reasoning modes. Local LLM support is a future optional provider for private/offline use, not a launch dependency.

We are building an ORVEX-owned modular realtime voice engine: streaming audio, VAD, endpointing, interruption/barge-in, AEC/noise handling, STT/TTS provider abstractions, and future custom voices. Permissions/tool authority remain outside the model.

Do not start by pasting code into chat. First summarize the non-negotiable architecture/workflow, inspect the uploaded V1 reference, and propose the single focused 0.0.1 foundation milestone. When implementing, use the patch ZIP -> root-safe apply script -> automated tests -> my live acceptance -> repair suffix if needed -> cleanup temporary patch/backup artifacts -> one focused commit workflow defined in the PDF. Challenge weak ideas instead of automatically agreeing with me; I make the final decision.

Appendix A. V1 fallback state

- Last confirmed accepted baseline in this handoff: 0.3.6 Permissions & Security committed after final Repair17 regression pass on September 10, 2026.
- 0.3.7 Autonomous Tasks was the next target and development had begun. Known active-context issue: "resume it" could resume YouTube instead of the discussed task. Treat 0.3.7 as experimental unless actual V1 Git history shows a later accepted commit.
- If Core v2 is abandoned or stalls, return to the current V1 ChatGPT Project and Jarvis_Real_Time folder. Do not attempt to reverse-merge V2 blindly into V1.

- Keep V1 runnable for a long time as test oracle/reference even after V2 becomes promising.

Appendix B. Patch cleanup command examples

Use exact filenames/folders printed by the installer. These are examples only; never use broad wildcards that may touch user data.

```
# Example PowerShell after acceptance
Remove-Item .\Jarvis_Core_v2_0.0.4_patch.zip -Force
Remove-Item .\apply_0.0.4_patch.py -Force
Remove-Item .\patch_files -Recurse -Force
Remove-Item .\<exact_timestamped_backup_folder> -Recurse -Force

git status --short
python -m unittest discover -s tests -v
git add .
git commit -m "0.0.4 - Conversation Core"
git push
```

DANGEROUS COMMAND WARNING

Never run `git clean -fd` casually in this project. It can delete untracked files such as `.env`, local databases, credentials, user data, and other important artifacts. Cleanup must be explicit and targeted.

Appendix C. Regression scenario inventory

- Wake word embedded in natural request; standalone "Jarvis" acknowledgement; sleep phrase.
- Interruption while Jarvis is speaking; noise mistaken for interruption; game/media audio present.
- Pause/resume task while YouTube is also paused; current conversational referent wins.
- Natural browser variants; Open Gmail routes web; tab open/close/fuzzy-title behavior; YouTube media state remains separate from task state.
- Gmail draft/edit/review/send/cancel/save and Calendar create/update/delete/conflict/attendees, including recipient resolution, hidden/visible confirmations, blocked-delete preflight, and Gmail follow-up.
- Computer open/close app/game/file/folder; aliases; force close only when appropriate; unit tests dry-run only.
- Permission Allow once / Always allow / Block persistence; prompt focus; voice-prompt behavior; conversational approval; purchase second-confirmation with item/total.
- Authentication approval auto-close or safe-close fallback; account sign-in/recovery/lockout/verification expiry/device identity.
- Provider timeout/retry without duplicate action; model escalation preserves context.
- Voice-device disconnect/reconnect; speaker output not transcribed through AEC; TTS cancellation on barge-in.
- Memory writes only after verified outcome; no hallucinated completion saved as fact.
- Background task pause/resume/cancel/restart recovery; conversation remains available; UI/orb restart does not destroy core/background state.

Appendix D. Current model-direction notes and future local-provider option

Launch candidate: GPT-5.6 Luna through OpenAIProvider, with provider-agnostic Jarvis reasoning modes and stronger cloud-model escalation for High/Extreme or unusually hard tasks. Current Luna API price snapshot (Sep 15, 2026): \$0.20/M input, \$0.02/M cached input, \$1.20/M output; pricing is external/configurable and must be rechecked before final billing decisions. Keep LocalProvider in the architecture for future Private/offline/enterprise-resilience use, but do not make local-model management a launch blocker. If local models are added later, installation must be zero-config and hardware-aware; production must not require LM Studio.